

DA6401: Introduction to Deep Learning

Module 2C - Optimizers with Adaptive Learning Rates

Ganapathy Krishnamurthi

Department of Data Science and Artificial Intelligence
IIT Madras

Optimizers Explored Till Now

- **Gradient Descent (GD)**

- Uses entire dataset for each update
- Update rule: $\theta_{t+1} = \theta_t - \eta \nabla_{\theta} J(\theta)$

- **Stochastic Gradient Descent (SGD)**

- Uses single example for each update
- Faster but noisy updates

- **Mini-batch Gradient Descent**

- Uses batch of examples
- Balance between GD and SGD

- **Momentum-based methods**

- SGD with Momentum, Nesterov Accelerated Gradient
- Accumulate velocity in consistent directions

Limitations of Fixed Learning Rate Optimizers

Key Problems:

① Same learning rate for all parameters

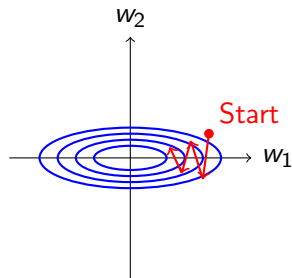
- Some features occur frequently
- Some features are sparse

② Choosing learning rate is difficult

- Too large: divergence
- Too small: slow convergence

③ Learning rate schedules are manual

- Require expert knowledge
- Problem-specific tuning



Oscillation in steep directions

Adaptive Learning Rate Methods

Core Idea

Adapt the learning rate for each parameter individually based on:

- History of gradients
- Frequency of updates
- Magnitude of updates

Benefits:

- Automatic learning rate tuning
- Per-parameter adaptation
- Better convergence

Methods we'll cover:

- 1 AdaGrad
- 2 RMSProp
- 3 AdaDelta

Adaptive Gradient Algorithm

AdaGrad

Adaptive Gradient Algorithm (Duchi et al., 2011)

Key Innovation:

- Accumulates squared gradients
- Larger accumulated gradients $\rightarrow$ smaller LR
- Smaller accumulated gradients $\rightarrow$ larger LR

Ideal for:

- Sparse data
- Features with different frequencies
- NLP tasks

Main Principle

“Parameters with large gradients should have their learning rate reduced, while parameters with small gradients should have their learning rate increased.”

AdaGrad: Intuition

Why adapt learning rates per parameter?

Example

Consider a word embedding model:

- Word "the" appears very frequently → Large gradients accumulated
- Word "aardvark" appears rarely → Small gradients accumulated

AdaGrad's approach:

- "the": Reduce learning rate (already learned well)
- "aardvark": Increase learning rate (needs more learning)

General Principle

- Frequently updated parameters → Smaller effective learning rate
- Infrequently updated parameters → Larger effective learning rate
- Automatically balances the learning process

AdaGrad: The Update Rule

Algorithm 1 AdaGrad Algorithm

- 1: **Initialize:** θ_0 , learning rate η , small constant ϵ (e.g., 10^{-8})
 - 2: **Initialize:** Gradient accumulator $G_0 = 0$
 - 3: **for** $t = 1$ to T **do**
 - 4: Compute gradient: $g_t = \nabla_{\theta} J(\theta_t)$
 - 5: Accumulate squared gradient: $G_t = G_{t-1} + g_t \odot g_t$
 - 6: Update parameters: $\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{G_t + \epsilon}} \odot g_t$
 - 7: **end for**
-

Key Points:

- G_t : Accumulates sum of squared gradients (element-wise)
- $\odot$: Element-wise multiplication
- ϵ : Prevents division by zero
- Effective learning rate: $\frac{\eta}{\sqrt{G_t + \epsilon}}$ (decreases over time)

Analyzing AdaGrad

For parameter θ_i at time t :

$$G_{t,i} = \sum_{\tau=1}^t (g_{\tau,i})^2$$
$$\theta_{t+1,i} = \theta_{t,i} - \frac{\eta}{\sqrt{G_{t,i} + \epsilon}} \cdot g_{t,i}$$

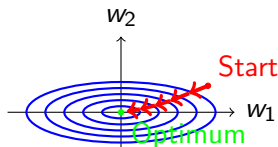
Advantages:

- No manual learning rate tuning
- Automatic per-parameter adaptation
- Excellent for sparse features
- Theoretical convergence guarantees

Observations:

- Learning rate monotonically decreases
- Early gradients have lasting impact
- Each parameter has its own learning rate
- G_t grows without bound

Visualizing the Effect of AdaGrad



AdaGrad adapts learning rate per dimension Larger steps in flat directions, smaller in steep directions

- Automatically scales learning rate for each dimension
- Reduces oscillations in steep directions
- Maintains progress in flat directions

Limitation of AdaGrad

Critical Problem: Aggressive Learning Rate Decay

Since $G_t = \sum_{\tau=1}^t g_{\tau}^2$, the accumulated gradient **grows monotonically**.

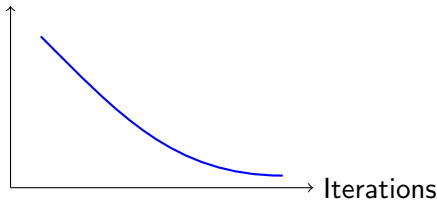
Consequence:

$$\lim_{t \rightarrow \infty} \frac{\eta}{\sqrt{G_t + \epsilon}} = \lim_{t \rightarrow \infty} \frac{\eta}{\sqrt{\sum_{\tau=1}^t g_{\tau}^2 + \epsilon}} \rightarrow 0$$

Problems:

- Learning rate eventually becomes infinitesimally small
- Training effectively stops
- Cannot escape local minima late in training

Effective LR



RMSProp Optimizer

Origin

Proposed by Geoffrey Hinton in his Coursera course (2012). Never published in a paper!

Key Innovation:

- Uses **exponential moving average** of squared gradients
- Recent gradients have more influence
- Past gradients decay exponentially
- Prevents aggressive decay of AdaGrad

Addresses AdaGrad's limitation:

- Maintains non-zero learning rate
- Adapts to recent gradient behavior
- Better for non-convex optimization
- Suitable for online/mini-batch learning

Main Idea: Keep a moving average of squared gradients instead of accumulating all of them

RMSProp: Intuition

Example

Consider training a neural network:

- **Early in training:** Gradients might be large and unstable
- **Later in training:** Gradients become smaller and more stable

Early large gradients permanently reduce learning rate. Forget old gradients gradually, focus on recent behavior

Exponential Moving Average

$$E[g^2]_t = \beta E[g^2]_{t-1} + (1 - \beta)g_t^2$$

where β (typically 0.9) controls the decay rate:

- Recent gradient g_t^2 has weight $(1 - \beta)$, Past average $E[g^2]_{t-1}$ has weight β
- Gradients from $\sim \frac{1}{1-\beta}$ steps ago have significant influence

RMSProp: The Update Rule

Algorithm 2 RMSProp Algorithm

- 1: **Initialize:** θ_0 , learning rate η , decay rate β (typically 0.9)
 - 2: **Initialize:** $E[g^2]_0 = 0$, small constant $\epsilon = 10^{-8}$
 - 3: **for** $t = 1$ to T **do**
 - 4: Compute gradient: $g_t = \nabla_{\theta} J(\theta_t)$
 - 5: Update moving average: $E[g^2]_t = \beta E[g^2]_{t-1} + (1 - \beta) g_t \odot g_t$
 - 6: Update parameters: $\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{E[g^2]_t + \epsilon}} \odot g_t$
 - 7: **end for**
-

Key Differences from AdaGrad:

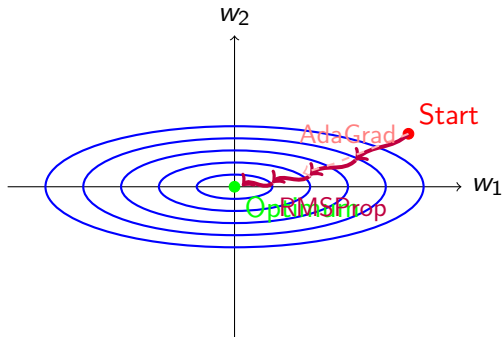
- **AdaGrad:** $G_t = G_{t-1} + g_t^2$ (unbounded accumulation)
- **RMSProp:** $E[g^2]_t = \beta E[g^2]_{t-1} + (1 - \beta) g_t^2$ (bounded by $\frac{1}{1-\beta} \max_{\tau} g_{\tau}^2$)

The learning rate now depends on a **weighted average** of recent squared gradients!

Comparison: RMSProp vs AdaGrad

Aspect	AdaGrad	RMSProp
Gradient accumulation	$G_t = \sum_{\tau=1}^t g_{\tau}^2$	$E[g^2]_t = \beta E[g^2]_{t-1} + (1-\beta)g_t^2$
Learning rate behavior	Monotonically decreases	Can increase or decrease
Long training	Stops learning	Continues learning
Memory of gradients	All history equally	Recent gradients emphasized
Best for	Sparse features, convex problems	Non-convex, online learning
Hyperparameter	η	η, β

Visualizing the Effect of RMSProp



**RMSProp maintains adaptive
learning throughout training**

Better convergence in non-convex landscapes

Limitation of RMSProp

Problem: Dependency on Initial Learning Rate

RMSProp still requires careful selection of the global learning rate η .

Issues:

① Learning rate sensitivity:

- Too large η : Can cause instability or divergence
- Too small η : Slow convergence
- Still needs manual tuning

② Unit inconsistency:

- Gradient g_t and learning rate have different units
- Update step: $\Delta\theta = \frac{\eta}{\sqrt{E[g^2]_t}} g_t$
- Units don't match parameters' natural scale

③ Hyperparameter tuning:

- Need to tune both η and β
- Different problems may need different values

AdaDelta: An Adaptive Learning Rate Method

Origin

Matthew D. Zeiler, 2012: "ADADELTA: An Adaptive Learning Rate Method"

Key Innovation:

- **No learning rate parameter needed!**
- Uses only exponential decay rates
- Corrects unit mismatch in RMSProp
- More robust to hyperparameters

Main Ideas:

- 1 Accumulate gradients like RMSProp
- 2 Also accumulate updates ($\Delta\theta$)
- 3 Use RMS of updates to scale learning
- 4 Automatic unit correction

Motivation: The units of $\Delta\theta$ should match the units of θ

AdaDelta: Overcoming the Initial LR Problem

$$\text{Gradient units: } [g] = \frac{\partial L}{\partial \theta}$$

$$\text{Parameter units: } [\theta]$$

$$\text{Update in RMSProp: } \Delta\theta = -\frac{\eta}{\sqrt{E[g^2]}}g$$

- $[\Delta\theta]$ should equal $[\theta]$ for consistency. But in RMSProp: $[\Delta\theta] = \frac{[\eta]}{[g]} \cdot [g] = [\eta]$

AdaDelta Solution: Use RMS of Parameter Updates

Replace η with $\text{RMS}[\Delta\theta]_{t-1}$:

$$\Delta\theta_t = -\frac{\text{RMS}[\Delta\theta]_{t-1}}{\text{RMS}[g]_t}g_t$$

$$\text{Now: } [\Delta\theta_t] = \frac{[\theta]}{[g]} \cdot [g] = [\theta] \quad \checkmark$$

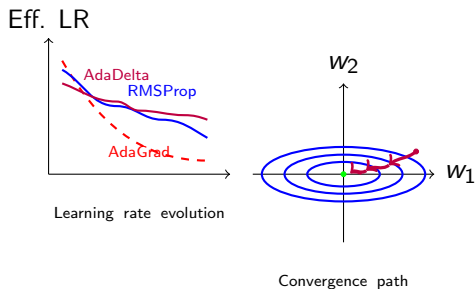
AdaDelta: The Update Rule

Algorithm 3 AdaDelta Algorithm

- 1: **Initialize:** θ_0 , decay rate ρ (typically 0.9 or 0.95)
 - 2: **Initialize:** $E[g^2]_0 = 0$, $E[\Delta\theta^2]_0 = 0$, $\epsilon = 10^{-6}$
 - 3: **for** $t = 1$ to T **do**
 - 4: Compute gradient: $g_t = \nabla_{\theta} J(\theta_t)$
 - 5: Accumulate gradient: $E[g^2]_t = \rho E[g^2]_{t-1} + (1 - \rho) g_t \odot g_t$
 - 6: Compute update: $\Delta\theta_t = -\frac{\sqrt{E[\Delta\theta^2]_{t-1} + \epsilon}}{\sqrt{E[g^2]_t + \epsilon}} \odot g_t$
 - 7: Accumulate updates: $E[\Delta\theta^2]_t = \rho E[\Delta\theta^2]_{t-1} + (1 - \rho) \Delta\theta_t \odot \Delta\theta_t$
 - 8: Update parameters: $\theta_{t+1} = \theta_t + \Delta\theta_t$
 - 9: **end for**
-

- **No learning rate η !** Only decay rate ρ
- Maintains two exponential moving averages: gradients and updates
- RMS = Root Mean Square: $\text{RMS}[x]_t = \sqrt{E[x^2]_t + \epsilon}$

Visualizing AdaDelta



Advantages of AdaDelta:

- No manual learning rate tuning required
- Robust to choice of ρ (typically 0.9-0.95 works well)
- Continues learning even after many iterations
- Unit-consistent updates

Summary: Adaptive Optimizers

Method	Key Idea	Advantages	Limitations
AdaGrad	Accumulate all squared gradients	Per-parameter LR, Great for sparse data	LR $\rightarrow 0$ and Stops learning
RMSProp	Exponential moving average of squared gradients	Continues learning, Non-convex friendly	Needs LR tuning and Unit mismatch
AdaDelta	Uses RMS of updates instead of LR	No LR needed, Unit consistent	Sometimes slower, One more hyperparameter

What's Missing?

These methods only consider **second moment** (variance) of gradients.
What about the **first moment** (mean/momentum)?

Adaptive Moment Estimation (Adam)

Adaptive Moment Estimation

Kingma & Ba, 2015: “Adam: A Method for Stochastic Optimization”

- **Momentum** (first moment)
- **RMSPprop** (second moment)

Additional Innovation:

- Bias correction for moments
- Especially important early in training

Why Adam?

- Most popular optimizer in deep learning
- Works well with minimal tuning
- Efficient computation
- Good default choice
- Combines momentum + adaptation

Two Key Components:

① First Moment (Mean) - Momentum:

- $m_t = \beta_1 m_{t-1} + (1 - \beta_1)g_t$
- Exponential moving average of gradients
- Provides direction and velocity
- Typical value: $\beta_1 = 0.9$

② Second Moment (Variance) - Adaptive LR:

- $v_t = \beta_2 v_{t-1} + (1 - \beta_2)g_t^2$
- Exponential moving average of squared gradients
- Controls step size per parameter
- Typical value: $\beta_2 = 0.999$

Intuition

- m_t : “Where should we go?” (direction with momentum)
- v_t : “How big should our steps be?” (adaptive scaling)

Adam: The Bias Correction

Since $m_0 = 0$ and $v_0 = 0$:

$$m_1 = \beta_1 \cdot 0 + (1 - \beta_1)g_1 = (1 - \beta_1)g_1$$

$$v_1 = \beta_2 \cdot 0 + (1 - \beta_2)g_1^2 = (1 - \beta_2)g_1^2$$

These are **biased toward zero**, especially when β_1, β_2 are close to 1!

Expected Values:

$$E[m_t] = E[g_t] \cdot (1 - \beta_1^t) \neq E[g_t]$$

$$E[v_t] = E[g_t^2] \cdot (1 - \beta_2^t) \neq E[g_t^2]$$

Bias Correction

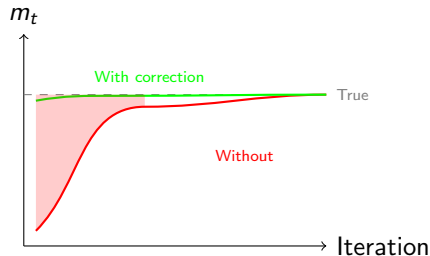
$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t} \Rightarrow E[\hat{m}_t] = E[g_t]$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t} \Rightarrow E[\hat{v}_t] = E[g_t^2]$$

Importance of Bias Correction

Without bias correction:

- Early iterations: moments severely underestimated
- With $\beta_1 = 0.9$: $m_1 = 0.1 \cdot g_1$
(90% underestimate!)
- With $\beta_2 = 0.999$: $v_1 = 0.001 \cdot g_1^2$
(99.9% underestimate!)
- Results in very large initial steps
- Can cause divergence



Bias correction stabilizes early training

With bias correction:

- Moments correctly estimated
- Stable initial training
- Better convergence

t	$\frac{1}{1-0.9^t}$	$\frac{1}{1-0.999^t}$
1	10.0	1000.0
10	1.5	100.5
100	1.0	10.5
1000	1.0	2.0

Adam: The Update Rule

Algorithm 4 Adam Algorithm

- 1: **Initialize:** θ_0 , stepsize α (default: 0.001)
 - 2: **Initialize:** $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$
 - 3: **Initialize:** $m_0 = 0$, $v_0 = 0$
 - 4: **for** $t = 1$ to T **do**
 - 5: Compute gradient: $g_t = \nabla_{\theta} J(\theta_t)$
 - 6: Update biased first moment: $m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$
 - 7: Update biased second moment: $v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$
 - 8: Compute bias-corrected first moment: $\hat{m}_t = \frac{m_t}{1 - \beta_1^t}$
 - 9: Compute bias-corrected second moment: $\hat{v}_t = \frac{v_t}{1 - \beta_2^t}$
 - 10: Update parameters: $\theta_{t+1} = \theta_t - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$
 - 11: **end for**
-

- Learning rate $\alpha = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$

NAdam Optimizer

Full Name & Origin

Nesterov-accelerated Adam

Dozat, 2016: "Incorporating Nesterov Momentum into Adam"

Key Innovation:

- Combines Adam with **Nesterov momentum**
- Look-ahead before computing gradient
- Often faster convergence than Adam

Recall Nesterov Momentum:

- Regular momentum: update from current position
- Nesterov: update from *future* position

Why NAdam?

- **Adam** gives adaptive learning rates
- **Nesterov** gives better momentum
- Combined: best of both worlds
- Especially good for computer vision

When to use:

- Vision tasks (CNNs)
- When you have compute budget

NAdam: Understanding the Difference

Standard Momentum:

- 1 Compute gradient at current position
- 2 Update velocity
- 3 Move in velocity direction

$$v_t = \beta v_{t-1} + (1 - \beta)g_t$$
$$\theta_{t+1} = \theta_t - \alpha v_t$$

Nesterov Momentum:

- 1 Look ahead using velocity
- 2 Compute gradient at *lookahead* position
- 3 Correct velocity, move in that direction

$$\tilde{\theta}_t = \theta_t + \beta v_{t-1}$$
$$g_t = \nabla J(\tilde{\theta}_t)$$
$$v_t = \beta v_{t-1} + (1 - \beta)g_t$$
$$\theta_{t+1} = \theta_t - \alpha v_t$$

Key Insight

Nesterov evaluates gradient at the *anticipated* position, allowing for better correction when overshooting.

NAdam: The Update Rule

Incorporating Nesterov into Adam:

Algorithm 5 NAdam Algorithm

- 1: **Initialize:** θ_0 , stepsize α (default: 0.002)
- 2: **Initialize:** $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$
- 3: **Initialize:** $m_0 = 0$, $v_0 = 0$
- 4: **for** $t = 1$ to T **do**
- 5: Compute gradient: $g_t = \nabla_{\theta} J(\theta_t)$
- 6: Update first moment: $m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$
- 7: Update second moment: $v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$
- 8: Bias-corrected moments: $\hat{m}_t = \frac{m_t}{1 - \beta_1^t}$, $\hat{v}_t = \frac{v_t}{1 - \beta_2^t}$
- 9: **Nesterov modification:** $\bar{m}_t = \beta_1 \hat{m}_t + \frac{(1 - \beta_1) g_t}{1 - \beta_1^t}$
- 10: Update parameters: $\theta_{t+1} = \theta_t - \alpha \frac{\bar{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$
- 11: **end for**

NAdam vs Adam: When to Use Each?

Aspect	Adam	NAdam
Momentum type	Standard momentum	Nesterov momentum
Convergence speed	Fast	Often faster
Typical performance	Excellent baseline	Sometimes better than Adam
Stability	Very stable	Stable, occasionally more sensitive
Best for	General purpose, default choice	CNNs, vision tasks, when you have compute time
Hyperparameters	Well-established defaults	Similar to Adam, sometimes needs slightly higher LR